

Capture the Flag 7: Stop Putting Everything on One Screen

Goal: Split your Student Directory into multiple screens using Flutter navigation, while using a modal for editing student information.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

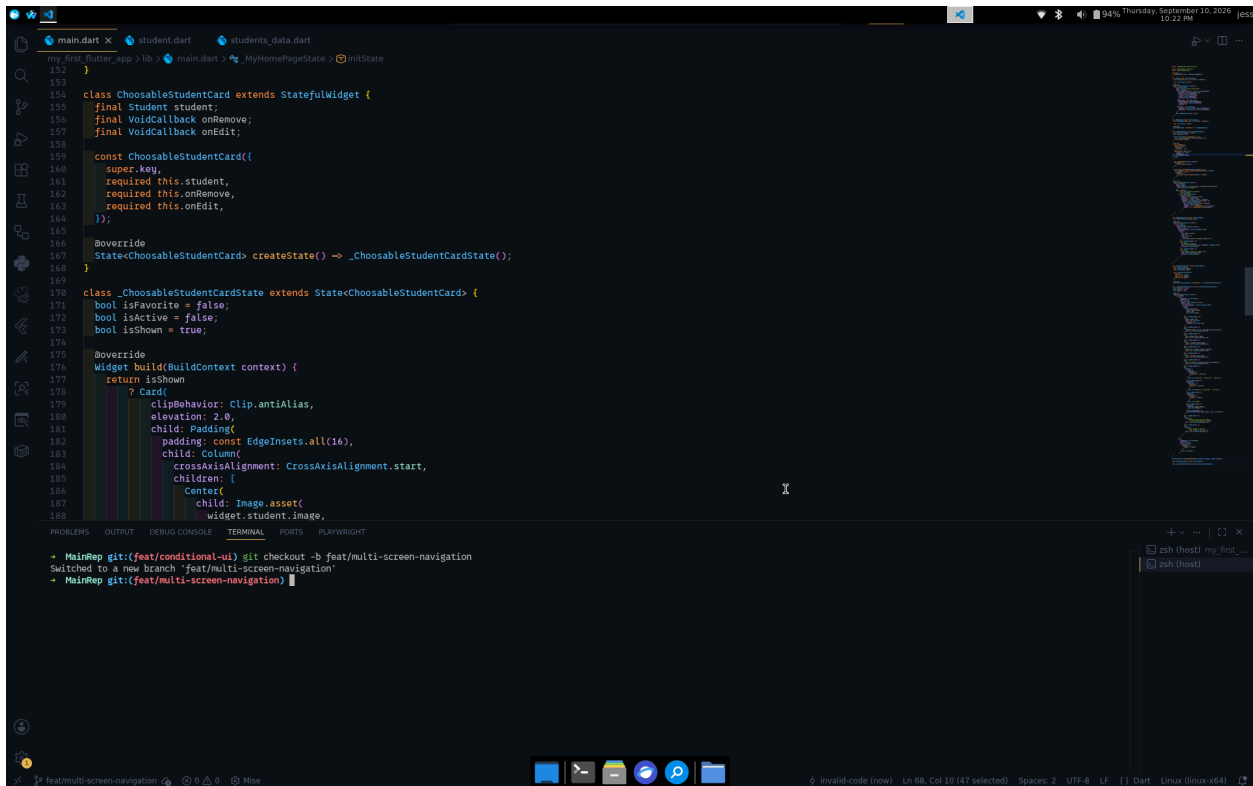
Objective: Create a new feature branch before adding multiple screens.

Create:

`feature/multi-screen-navigation`

Screenshot:

- Terminal showing your current branch



```
152 }
153
154 class ChoosableStudentCard extends StatefulWidget {
155   final Student student;
156   final VoidCallback onRemove;
157   final VoidCallback onEdit;
158
159   const ChoosableStudentCard({
160     super.key,
161     required this.student,
162     required this.onRemove,
163     required this.onEdit,
164   });
165
166   @override
167   State<ChoosableStudentCard> createState() => _ChoosableStudentCardState();
168
169   class _ChoosableStudentCardState extends State<ChoosableStudentCard> {
170     bool isFavorite = false;
171     bool isActive = false;
172     bool isShown = true;
173
174     @override
175     Widget build(BuildContext context) {
176       return isShown
177         ? Card(
178           clipBehavior: Clip.antiAlias,
179           elevation: 2.0,
180           child: Padding(
181             padding: const EdgeInsets.all(16),
182             child: Column(
183               crossAxisAlignment: CrossAxisAlignment.start,
184               children: [
185                 Center(
186                   child: Image.asset(
187                     widget.student.image,
```

🚩 Flag 1 — Create the Student List Screen

Objective: Move your Student Directory into its own screen.

Create a dedicated **Student List Screen** containing your existing student list.

Research how Flutter creates separate screens using **Widget** classes.

Test

The Student Directory should function as its own screen.

Screenshot:

- Student List Screen
- Student List screen code
- Application showing the screen

🚩 Flag 2 — Create the Student Details Screen

Objective: Create a separate screen for viewing a student's information.

When a student is selected, navigate:

Student List → Student Details

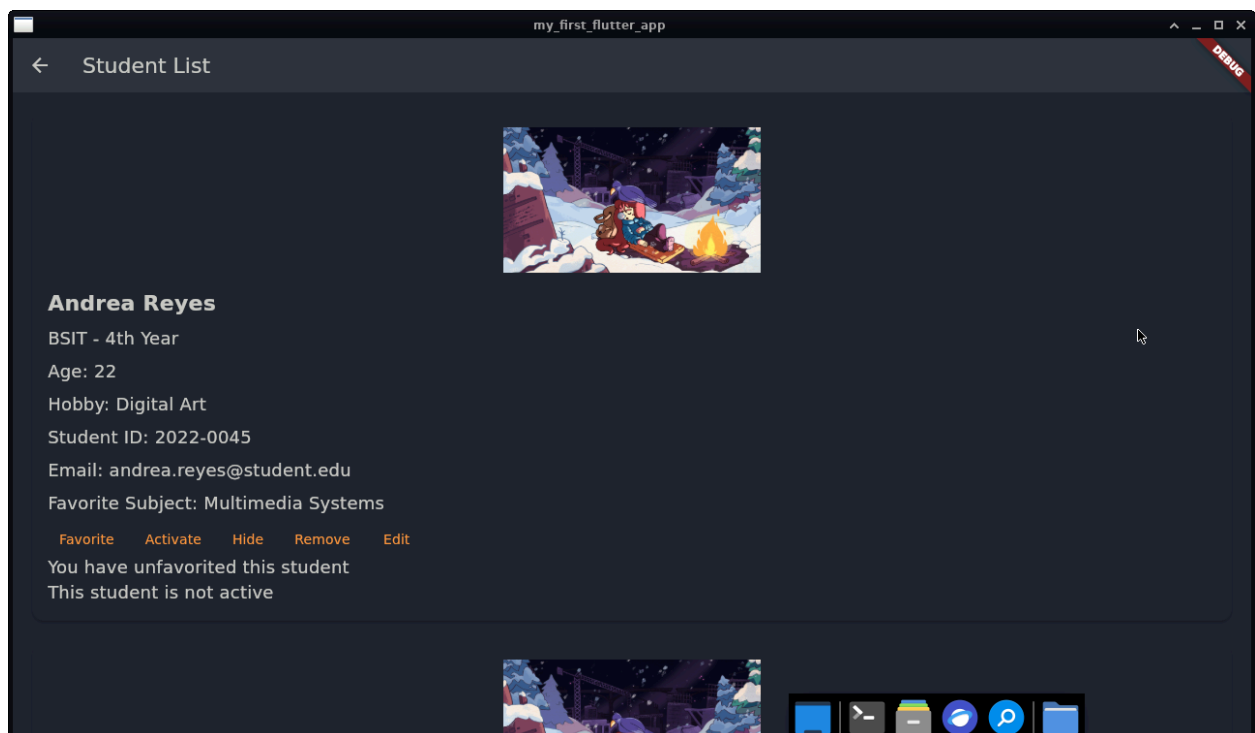
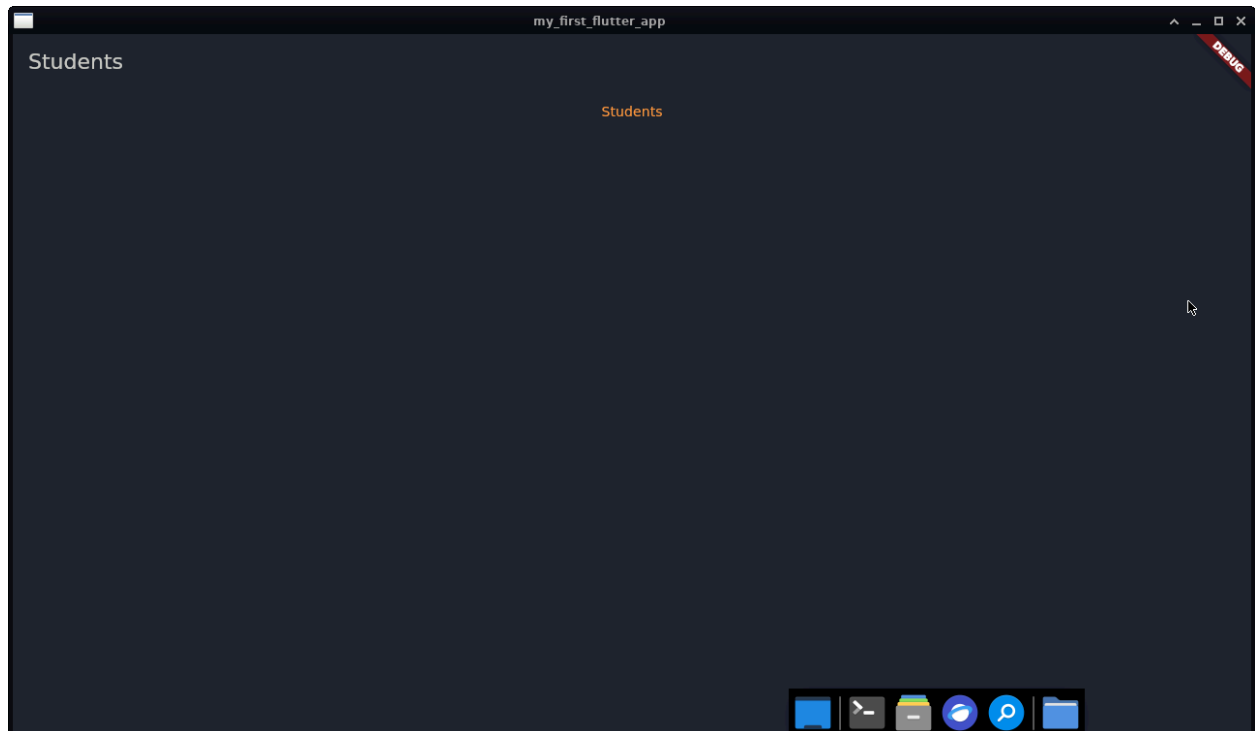
Research Flutter's **Navigator** and how to move between screens.

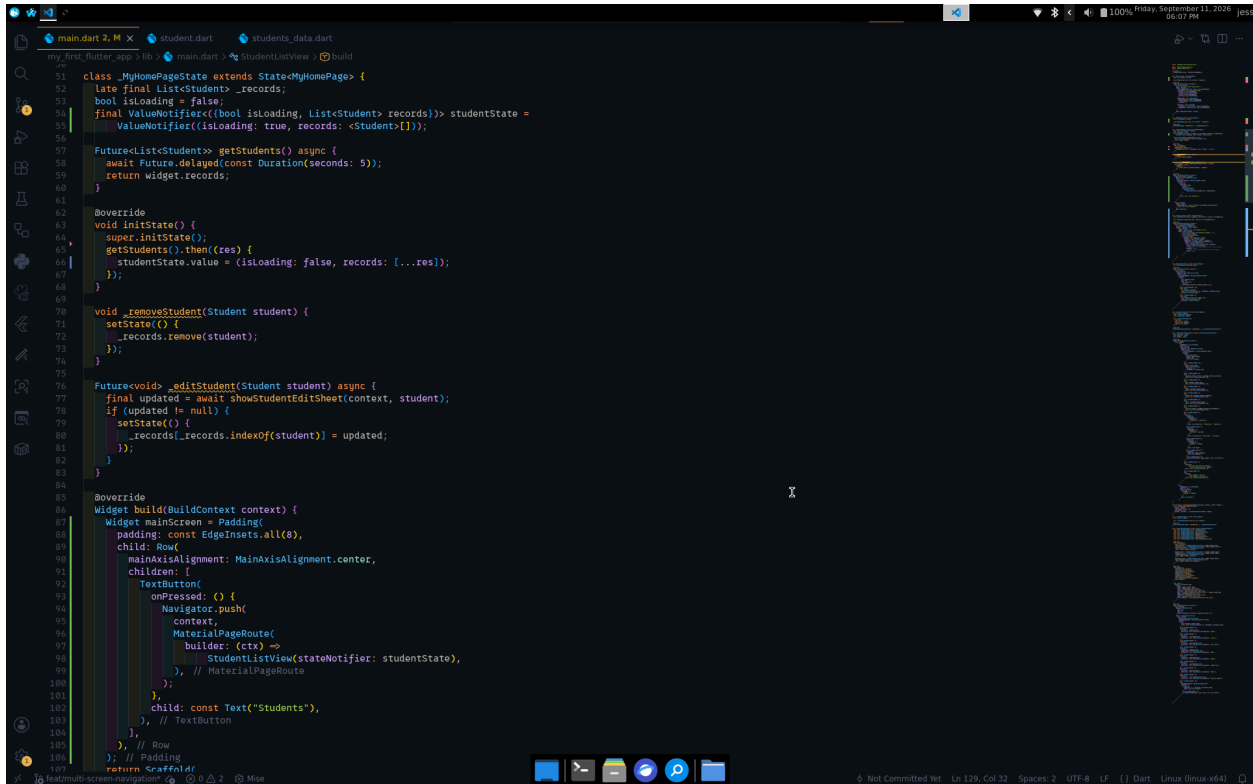
Test

1. Select a student.
2. Open the Student Details Screen. (No data displayed yet)
3. Return to the Student List using back navigation.

Screenshot:

- Student List Screen
- Student Details Screen
- Navigation code





```
51 class _MyHomePageState extends State<MyHomePage> {
52   late final List<Student> _records;
53   bool isLoading = false;
54   final ValueNotifier<(bool isLoading, List<Student> records)> studentState =
55     ValueNotifier<(bool isLoading, List<Student> records)>((false, <Student>[]));
56
57   Future<List<Student>> getStudents() async {
58     await Future.delayed(const Duration(seconds: 5));
59     return widget.records;
60   }
61
62   @override
63   void initState() {
64     super.initState();
65     getStudents().then((res) {
66       studentState.value = (isLoading: false, records: [...res]);
67     });
68   }
69
70   void _removeStudent(Student student) {
71     setState(() {
72       _records.remove(student);
73     });
74   }
75
76   Future<void> _editStudent(Student student) async {
77     final updated = await showStudentEditSheet(context, student);
78     if (updated != null) {
79       setState(() {
80         _records[_records.indexOf(student)] = updated;
81       });
82     }
83   }
84
85   @override
86   Widget build(BuildContext context) {
87     Widget mainScreen = Padding(
88       padding: const EdgeInsets.all(8),
89       child: Row(
90         mainAxisAlignment: MainAxisAlignment.center,
91         children: [
92           TextButton(
93             onPressed: () {
94               Navigator.push(
95                 context,
96                 MaterialPageRoute(
97                   builder: (ctx) =>
98                     StudentListView(stateNotifier: studentState),
99                 ), // MaterialPageRoute
100             ),
101         ],
102         child: const Text("Students"),
103       ), // Row
104     ), // Row
105   ); // Row
106   return Scaffold(
107     // ...
108   );
109 }
```

🚩 Flag 3 — Pass Student Data

Objective: Pass the selected student's data to the Student Details Screen.

The Details Screen should display the correct information for the selected student.

Display relevant information such as:

- Name
- Student ID
- Course
- Status
- Other existing student data

Research how data can be passed between Flutter screens.

Test

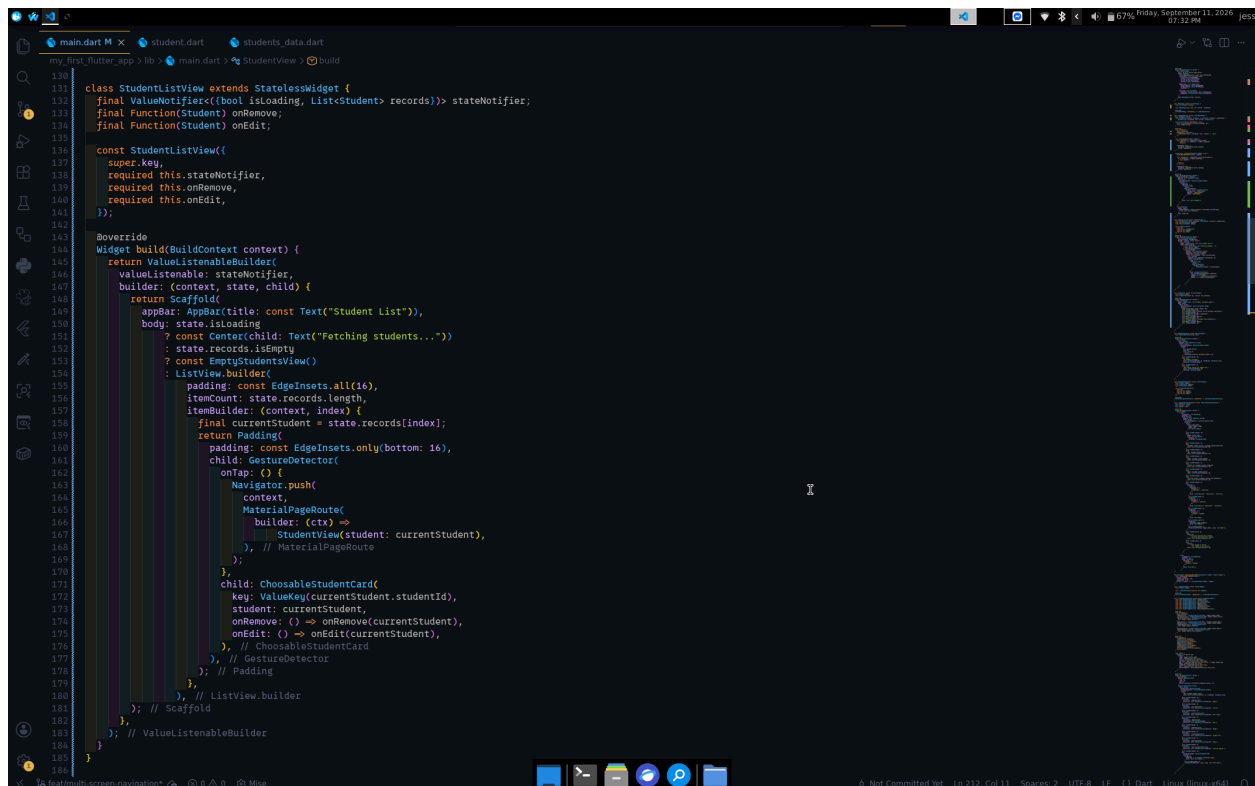
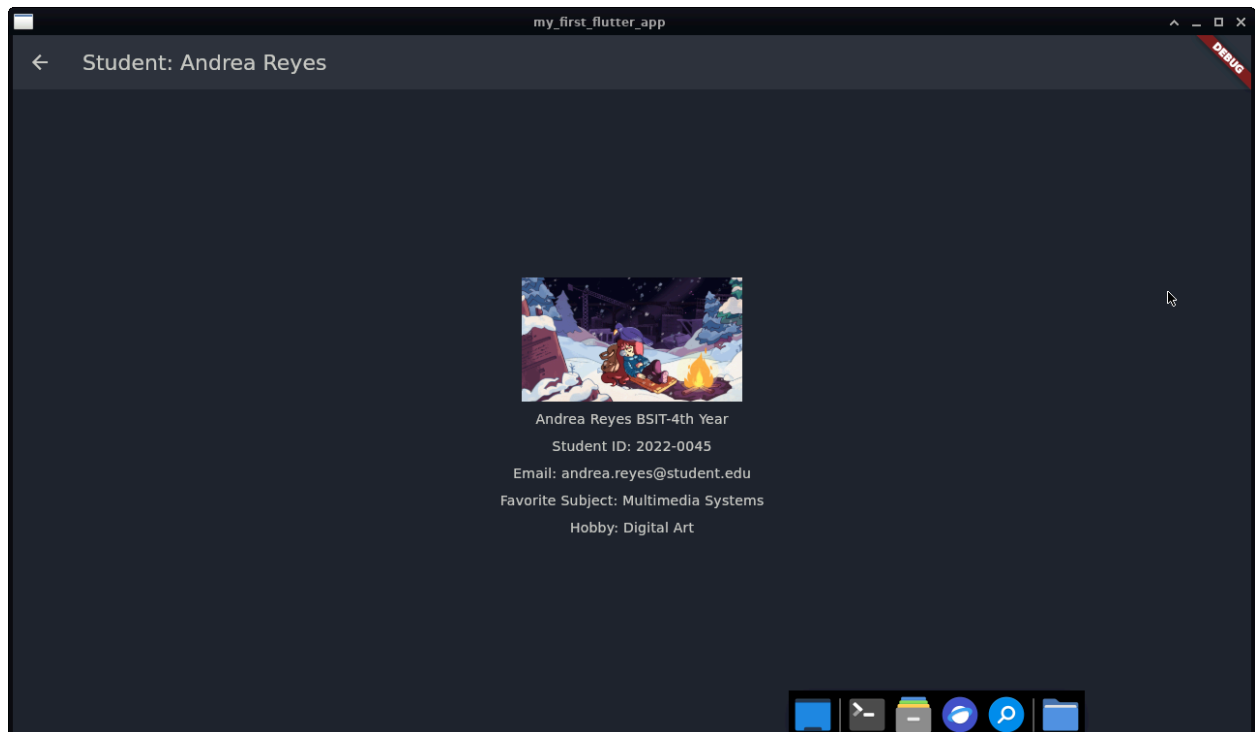
Selecting different students should display their corresponding information.

Student A → Student A's Details

Student B → Student B's Details

Screenshot:

- Selected student
- Student Details
- Data-passing code



Flag 4 — Edit with a Modal

Objective: Allow administrators to edit student information without creating another screen.

Add an **Edit** action to the Student Details Screen.

When selected, display an editing interface using a **modal**, such as a dialog or bottom sheet.

Research Flutter's:

- `showDialog`
- `showModalBottomSheet`

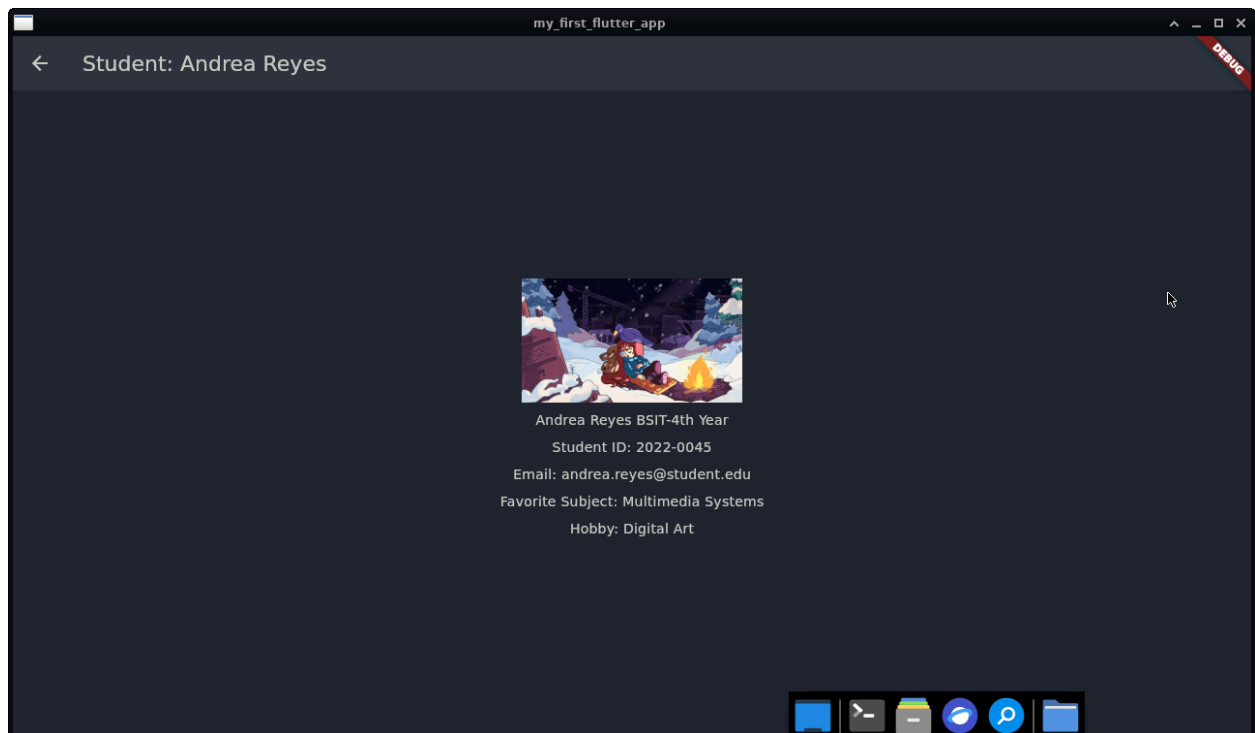
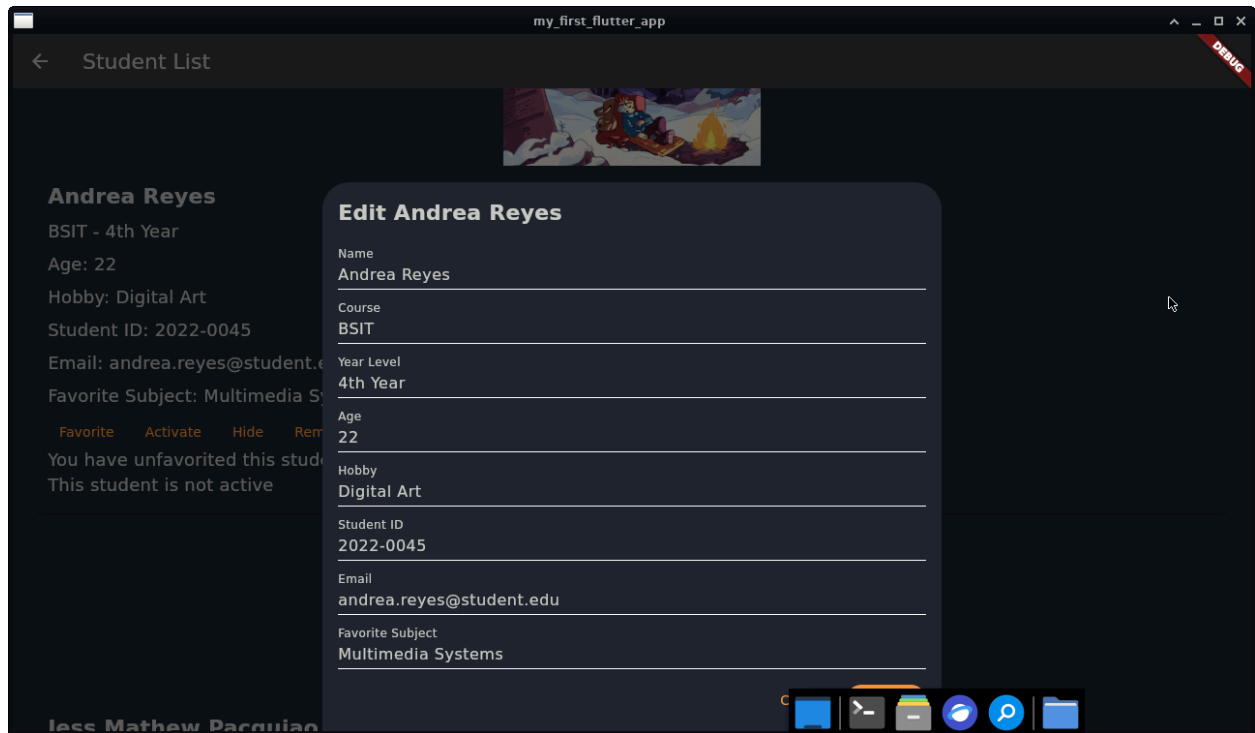
You may choose whichever approach is appropriate for your design.

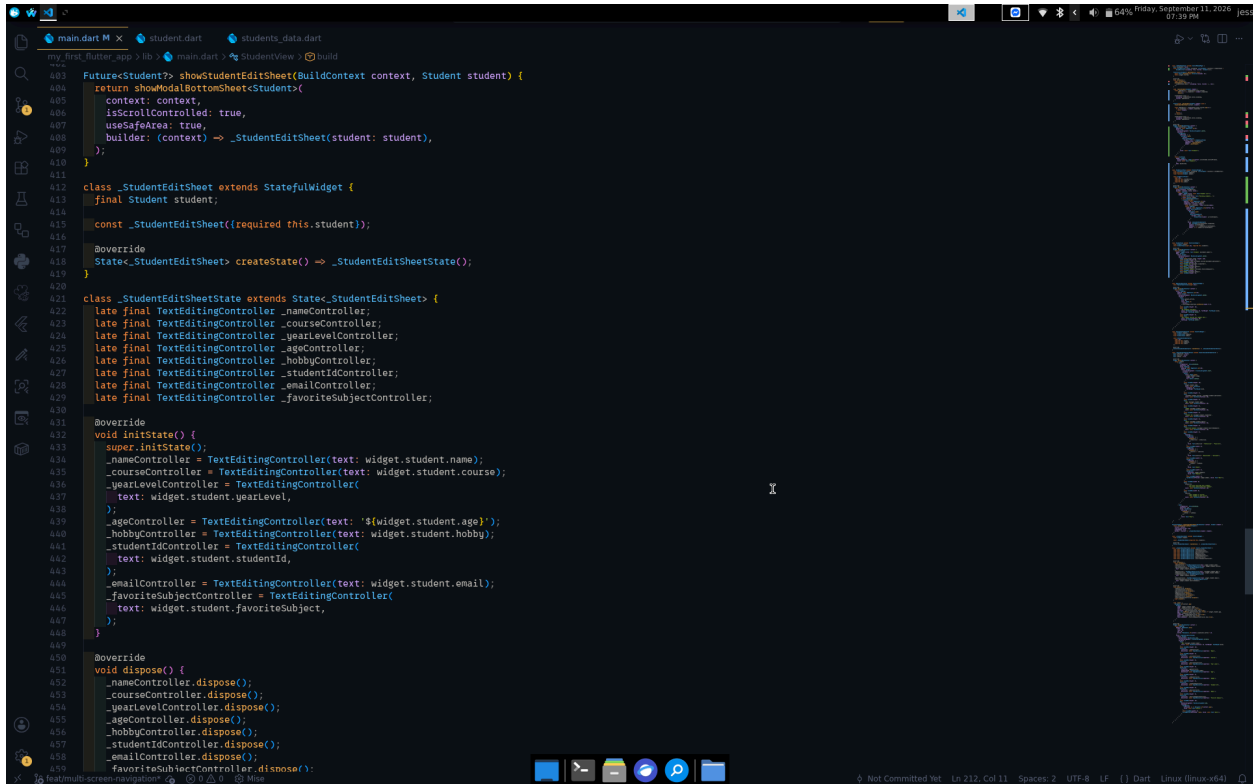
Test

1. Open Student Details.
2. Select Edit.
3. An editing modal should appear.
4. The modal should display the appropriate student's information.
5. Close or submit the modal.

Screenshot:

- Student Details Screen
- Edit modal
- Modal code





```
403 Future<Student?> showStudentEditSheet(BuildContext context, Student student) {
404   return showModalBottomSheet<Student>({
405     context: context,
406     isScrollControlled: true,
407     useSafeArea: true,
408     builder: (context) => _StudentEditSheet(student: student),
409   });
410 }
411
412 class _StudentEditSheet extends StatefulWidget {
413   final Student student;
414
415   const _StudentEditSheet({required this.student});
416
417   @override
418   State<_StudentEditSheet> createState() => _StudentEditSheetState();
419 }
420
421 class _StudentEditSheetState extends State<_StudentEditSheet> {
422   late final TextEditingController _nameController;
423   late final TextEditingController _courseController;
424   late final TextEditingController _yearLevelController;
425   late final TextEditingController _ageController;
426   late final TextEditingController _hobbyController;
427   late final TextEditingController _studentIdController;
428   late final TextEditingController _emailController;
429   late final TextEditingController _favoriteSubjectController;
430
431   @override
432   void initState() {
433     super.initState();
434     _nameController = TextEditingController(text: widget.student.name);
435     _courseController = TextEditingController(text: widget.student.course);
436     _yearLevelController = TextEditingController(
437       text: widget.student.yearLevel,
438     );
439     _ageController = TextEditingController(text: '${widget.student.age}');
440     _hobbyController = TextEditingController(text: widget.student.hobby);
441     _studentIdController = TextEditingController(
442       text: widget.student.studentId,
443     );
444     _emailController = TextEditingController(text: widget.student.email);
445     _favoriteSubjectController = TextEditingController(
446       text: widget.student.favoriteSubject,
447     );
448   }
449
450   @override
451   void dispose() {
452     _nameController.dispose();
453     _courseController.dispose();
454     _yearLevelController.dispose();
455     _ageController.dispose();
456     _hobbyController.dispose();
457     _studentIdController.dispose();
458     _emailController.dispose();
459     _favoriteSubjectController.dispose();
460   }
461 }
```

🚩 Flag 5 — Create the Add Student Screen

Objective: Create a separate screen for adding a new student.

Add an action from the Student List that navigates to:

Student List → Add Student

Create the **Add Student Screen**.

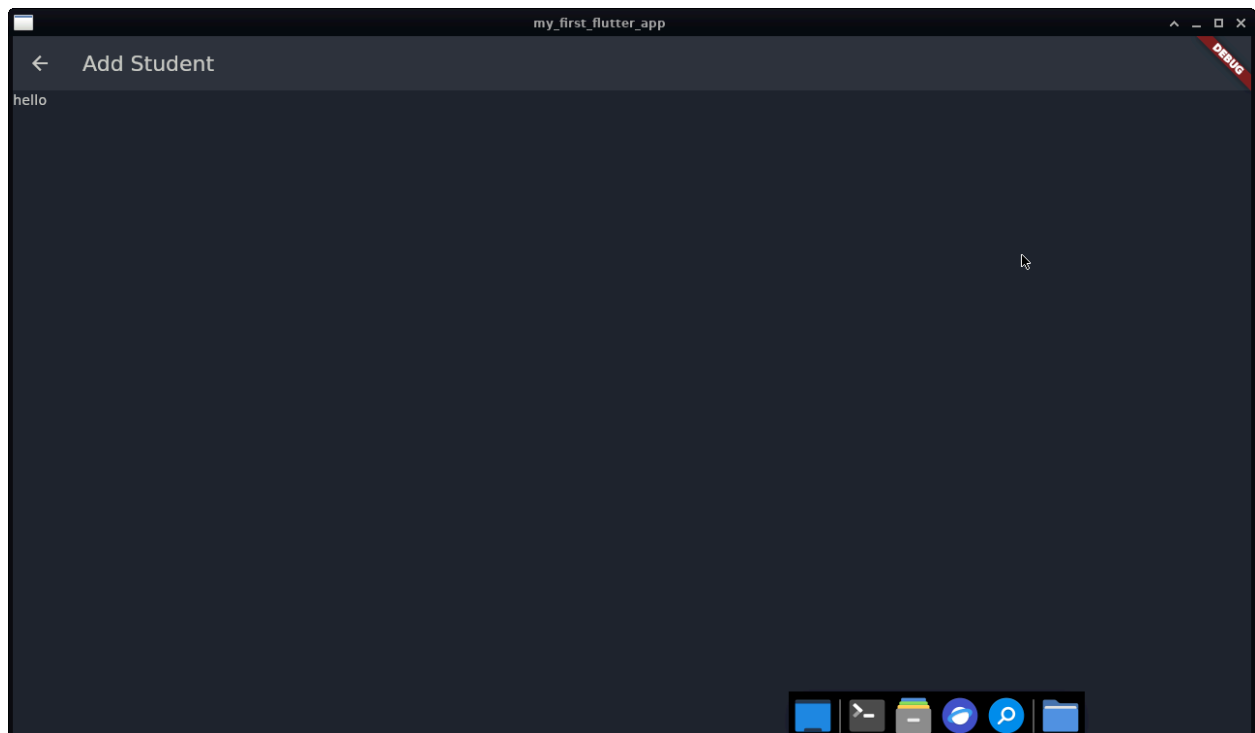
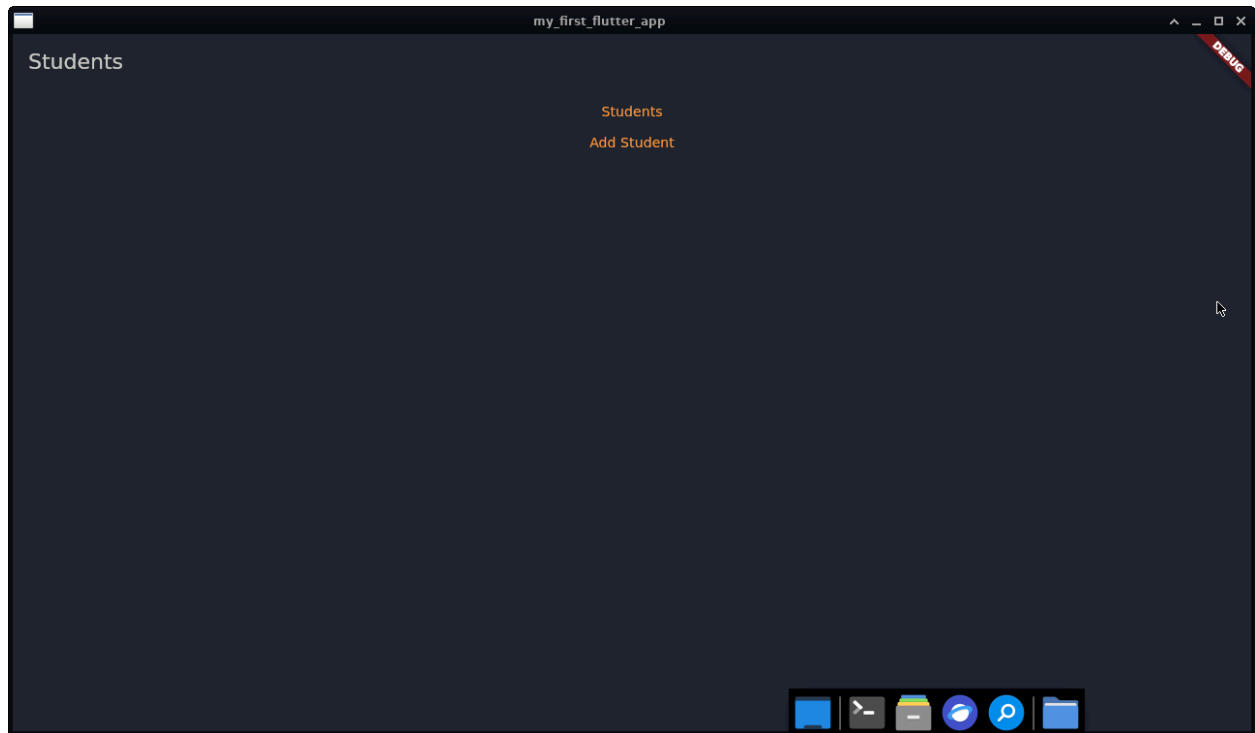
For now, focus on creating the screen and navigation structure.

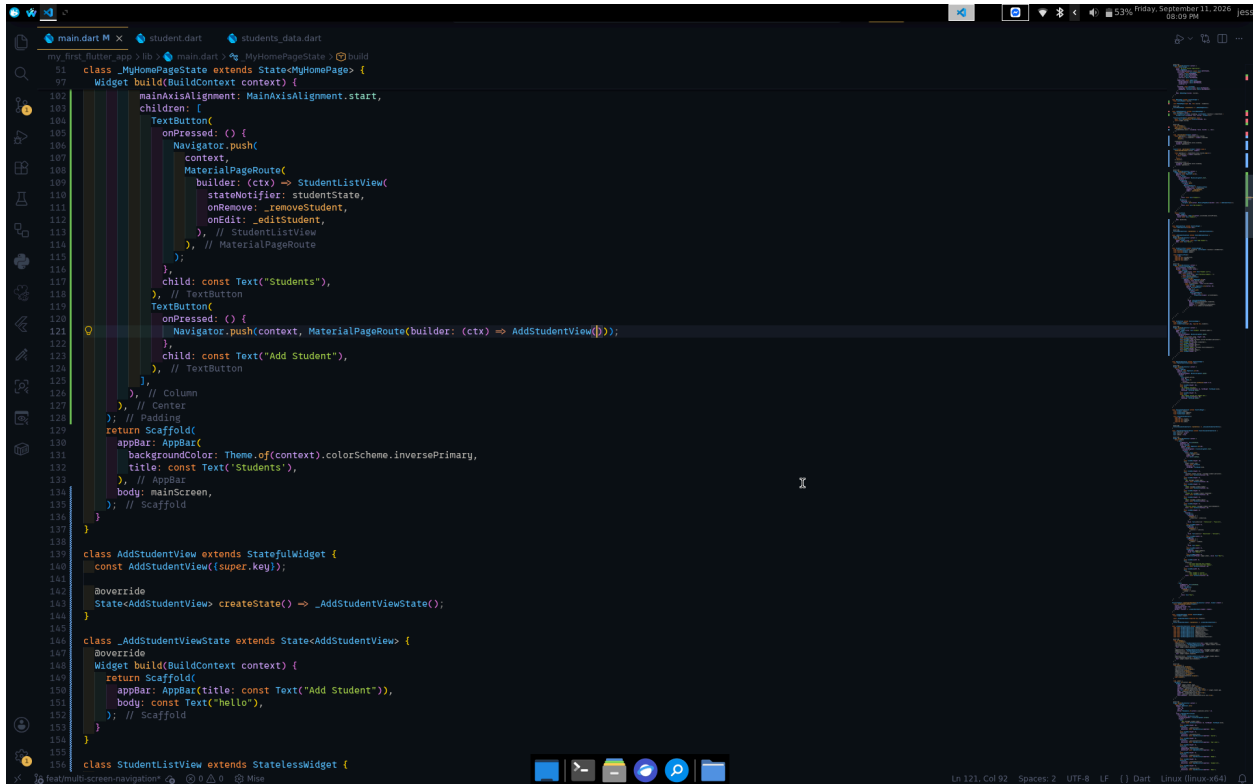
Test

Selecting Add Student should open the Add Student Screen.

Screenshot:

- Student List with Add action
- Add Student Screen
- Navigation code





```
51 class _MyHomePageState extends State<MyHomePage> {  
97   Widget build(BuildContext context) {  
102     mainAxisAlignment: MainAxisAlignment.start,  
103     children: [  
104       TextButton(  
105         onPressed: () {  
106           Navigator.push(  
107             context,  
108             MaterialPageRoute(  
109               builder: (ctx) => StudentListView(  
110                 stateNotifier: studentState,  
111                 onRemove: _removeStudent,  
112                 onEdit: _editStudent,  
113               ), // StudentListView  
114             ), // MaterialPageRoute  
115           ),  
116         ],  
117         child: const Text("Students"),  
118       ), // TextButton  
119       TextButton(  
120         onPressed: () {  
121           Navigator.push(context, MaterialPageRoute(builder: (ctx) => AddStudentView()));  
122         },  
123         child: const Text("Add Student"),  
124       ), // TextButton  
125     ],  
126   ), // Column  
127 ), // Center  
128 ); // Padding  
129 return Scaffold(  
130   appBar: AppBar(  
131     backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
132     title: const Text("Students"),  
133   ), // AppBar  
134   body: mainScreen,  
135 ); // Scaffold  
136 }  
137 }  
138  
139 class AddStudentView extends StatefulWidget {  
140   const AddStudentView({super.key});  
141  
142   @override  
143   State<AddStudentView> createState() => _AddStudentViewState();  
144 }  
145  
146 class _AddStudentViewState extends State<AddStudentView> {  
147   @override  
148   Widget build(BuildContext context) {  
149     return Scaffold(  
150       appBar: AppBar(title: const Text("Add Student")),  
151       body: const Text("hello"),  
152     ); // Scaffold  
153   }  
154 }  
155  
156 class StudentListView extends StatelessWidget {
```

🚩 Flag 6 — Prepare the Add Student Screen

Objective: Prepare the Add Student Screen for future functionality.

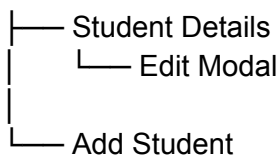
Organize the screen so it is ready to receive student input.

Do not implement the student input fields yet.

The screen should simply exist as a separate destination.

Your application should now have:

Student List

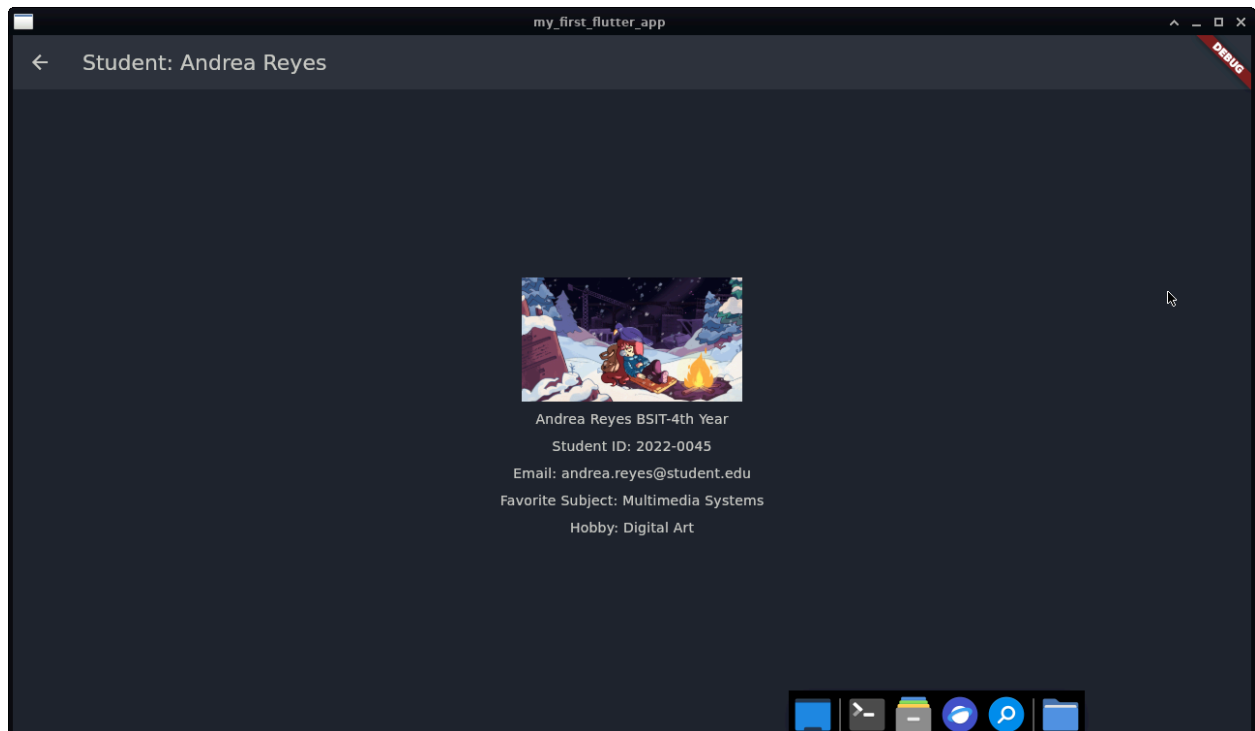
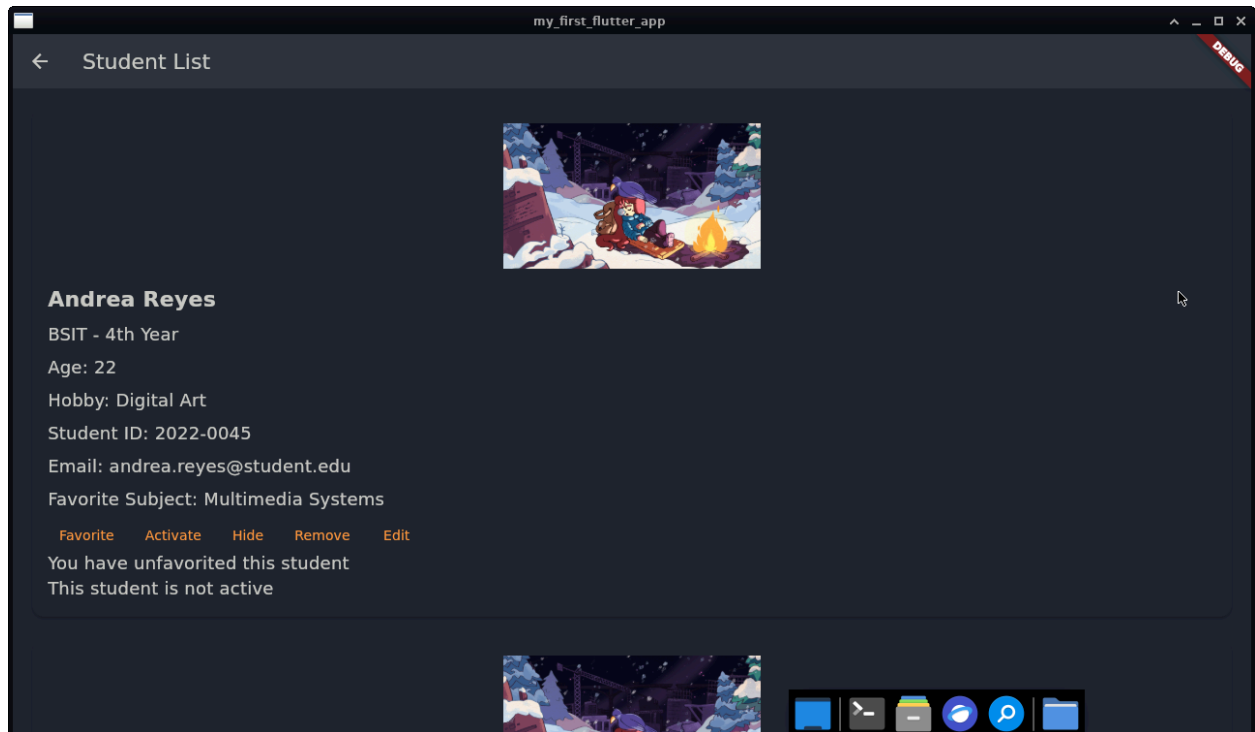


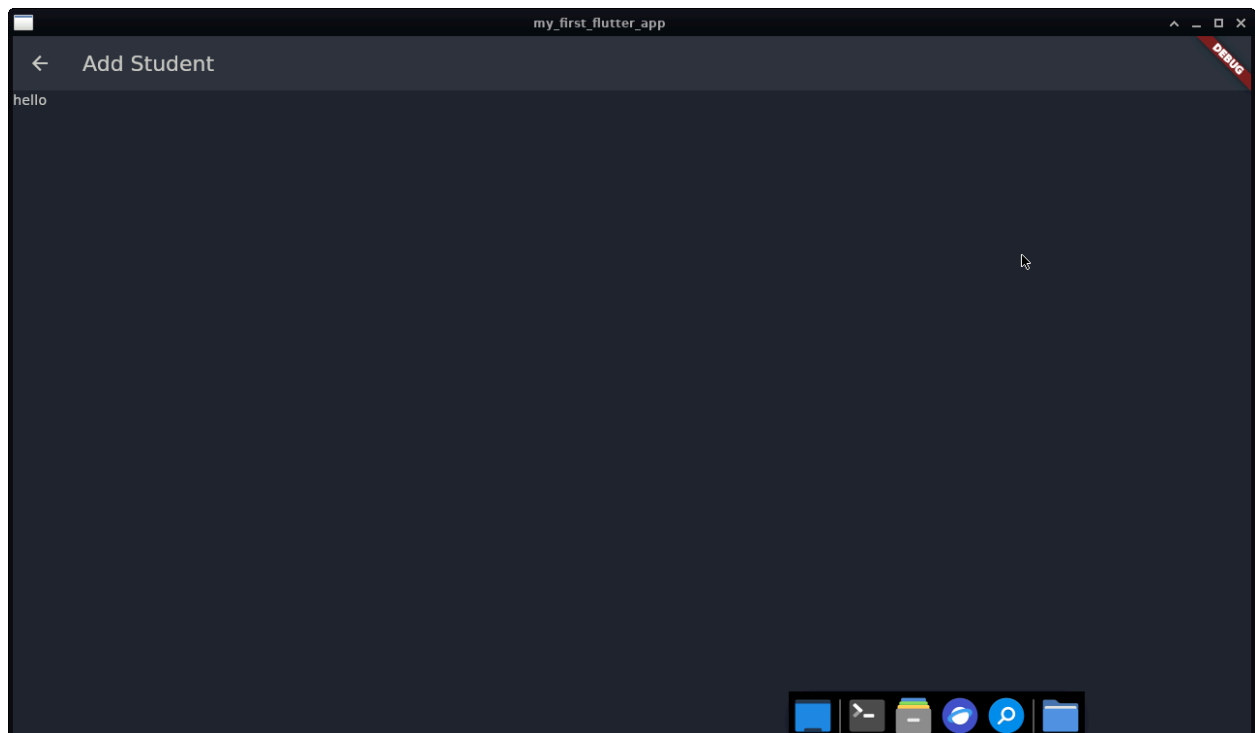
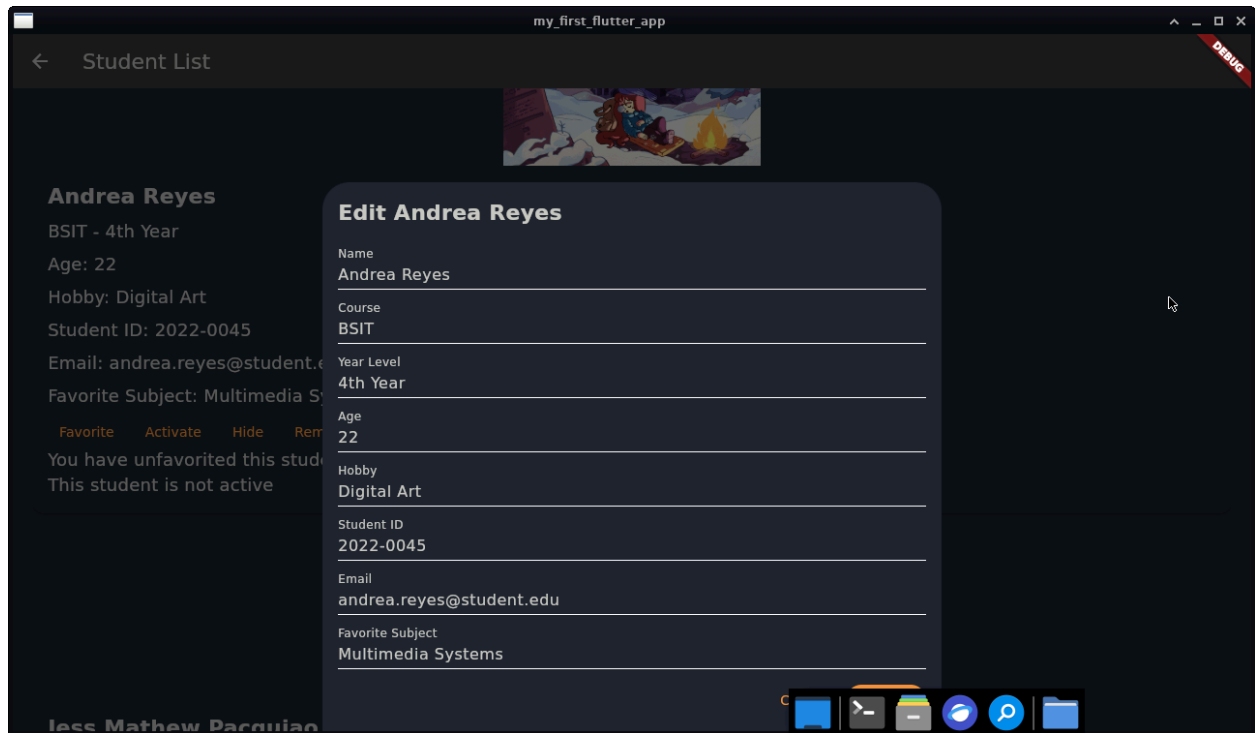
Test

You should be able to navigate between the appropriate screens without placing everything on one screen.

Screenshot:

- Student List
- Student Details
- Edit Modal
- Add Student Screen





🚩 Flag 7 — Connect the Routes

Objective: Organize the application's screens using Flutter routes.

Create routes for the major screens:

/student-list
/student-details
/add-student

Research:

- Named routes
- `Navigator.pushNamed`
- Passing arguments through routes

The **Edit Modal** should not be a route.

Test

Verify that:

- Student List opens correctly.
- Student Details opens correctly.
- The correct student data is passed.
- Add Student opens correctly.
- Edit opens as a modal from Student Details.

Screenshot:

- Route configuration
- Navigation using routes
- Student Details receiving student data
- Edit modal

```
main.dart M X student.dart X students_data.dart X
my_first_flutter_app > lib > main.dart > StudentListView > build

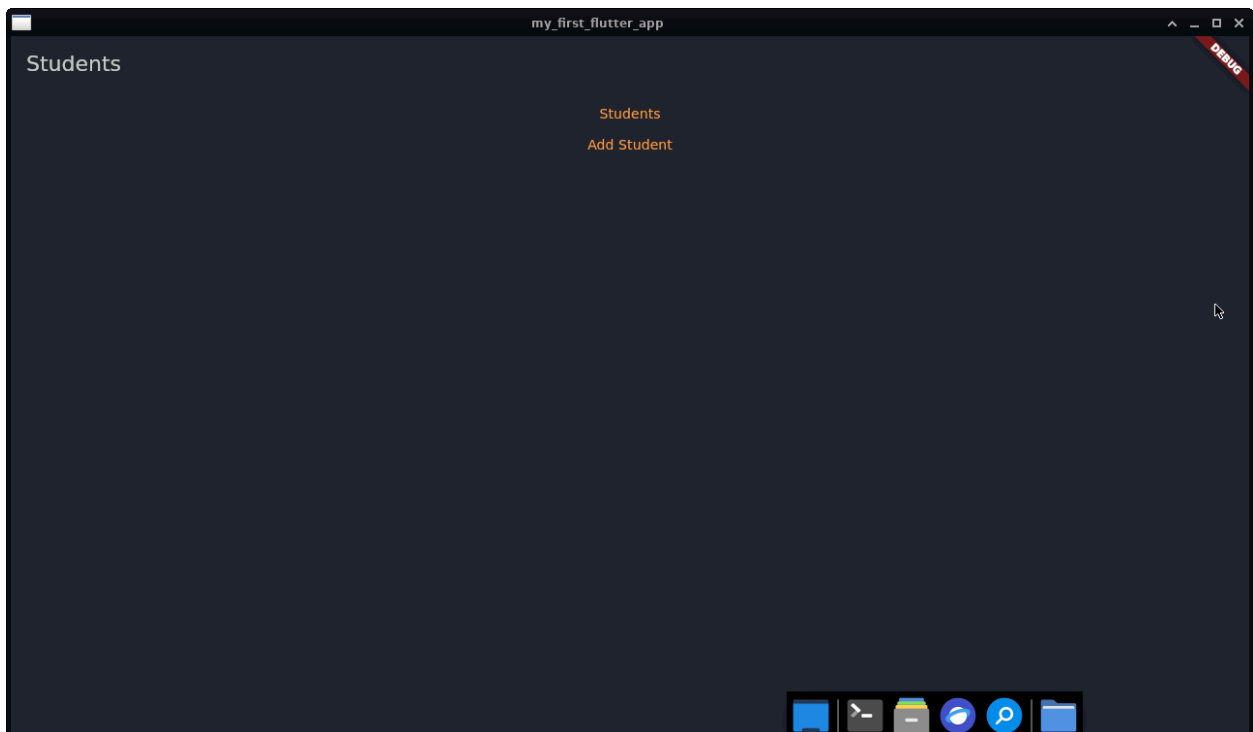
10 class MyApp extends StatelessWidget {
11   Widget build(BuildContext context) {
20     primary: Color(0xFFFF994D),
21     onPrimary: Color(0xFF1F2430),
22     surface: Color(0xFF1F2430),
23     onSurface: Color(0xFFC8C8C8),
24   }, // ColorsScheme.dark
25   appBarTheme: const AppBarTheme(
26     backgroundColor: Color(0xFF2D3640),
27     foregroundColor: Color(0xFFC8C8C8),
28     elevation: 0,
29   ), // AppBarTheme
30   textTheme: const TextTheme(
31     bodyLarge: TextStyle(color: Color(0xFFC8C8C8)),
32     bodyMedium: TextStyle(color: Color(0xFFC8C8C8)),
33   ), // TextTheme
34 }, // ThemeData
35 routes: {
36   "/": (context) => MyHomePage(records: records),
37   "/student-list": (context) => StudentListView(),
38   "/add-student": (context) => AddStudentView(),
39   "/student-details": (context) => const StudentView(),
40 },
41 ); // MaterialApp
42 }
43 }
44 }
45 }
46 }
47 class MyHomePage extends StatefulWidget {
48   final List<Student> records;
49   const MyHomePage({super.key, this.records = students});
50   @override
51   State<MyHomePage> createState() => _MyHomePageState();
52 }
53 }
54 }
55 }
```

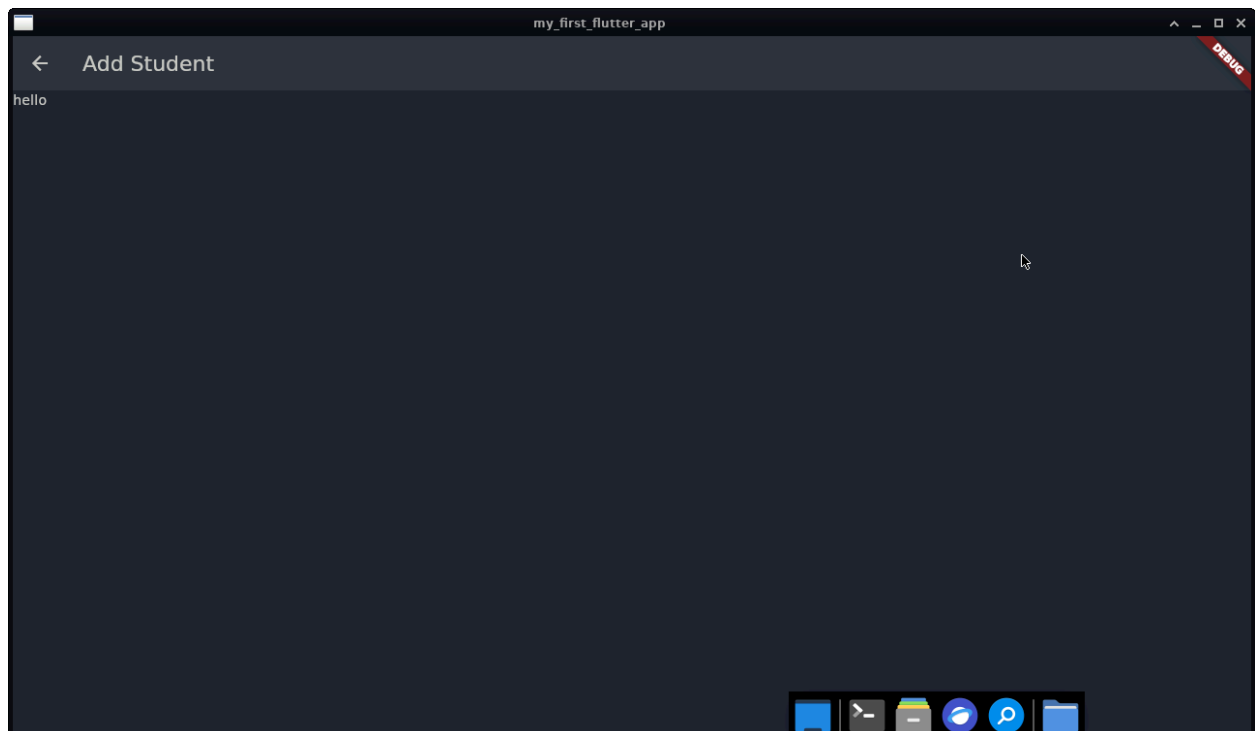
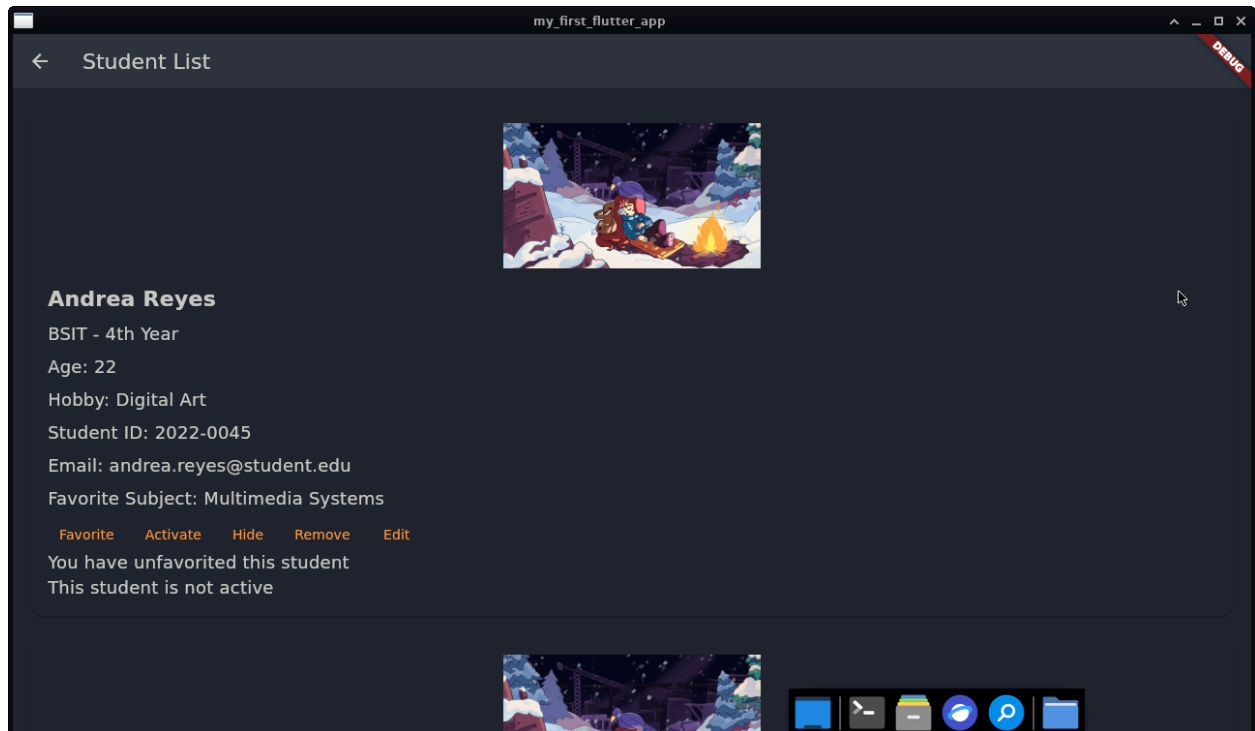
Problems Output Debug Console Terminal Ports Playwright

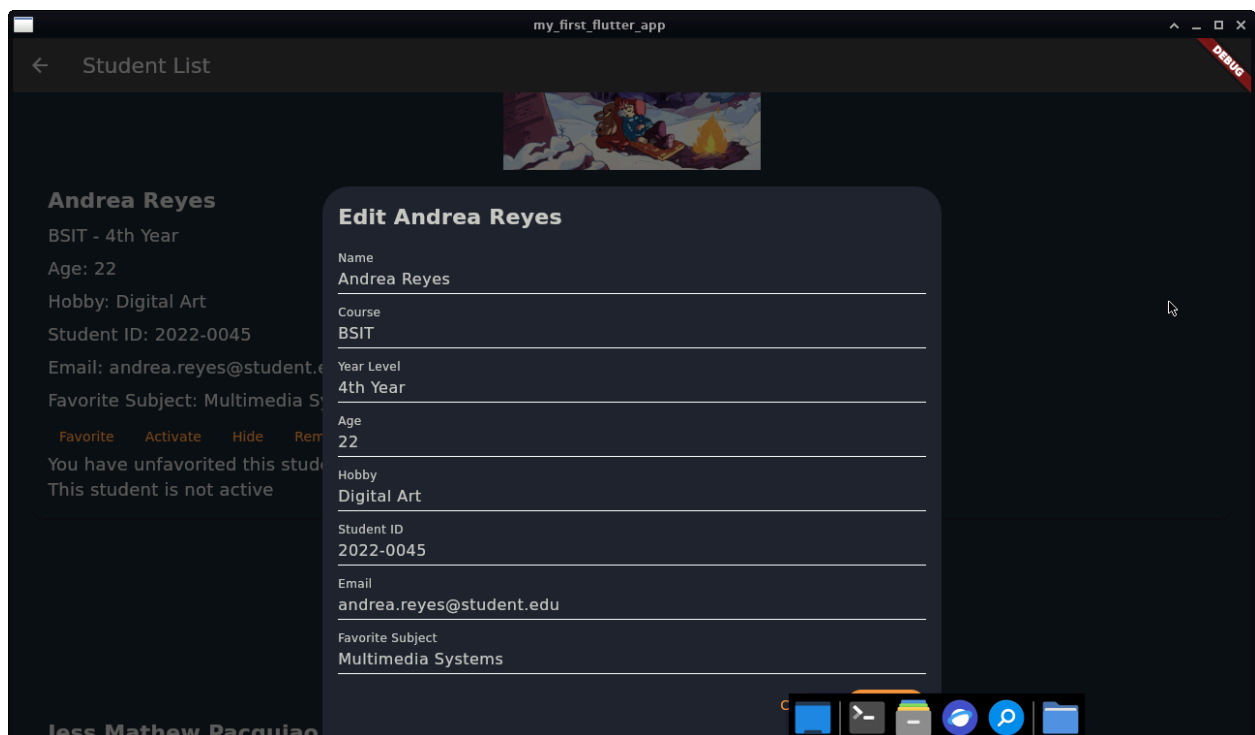
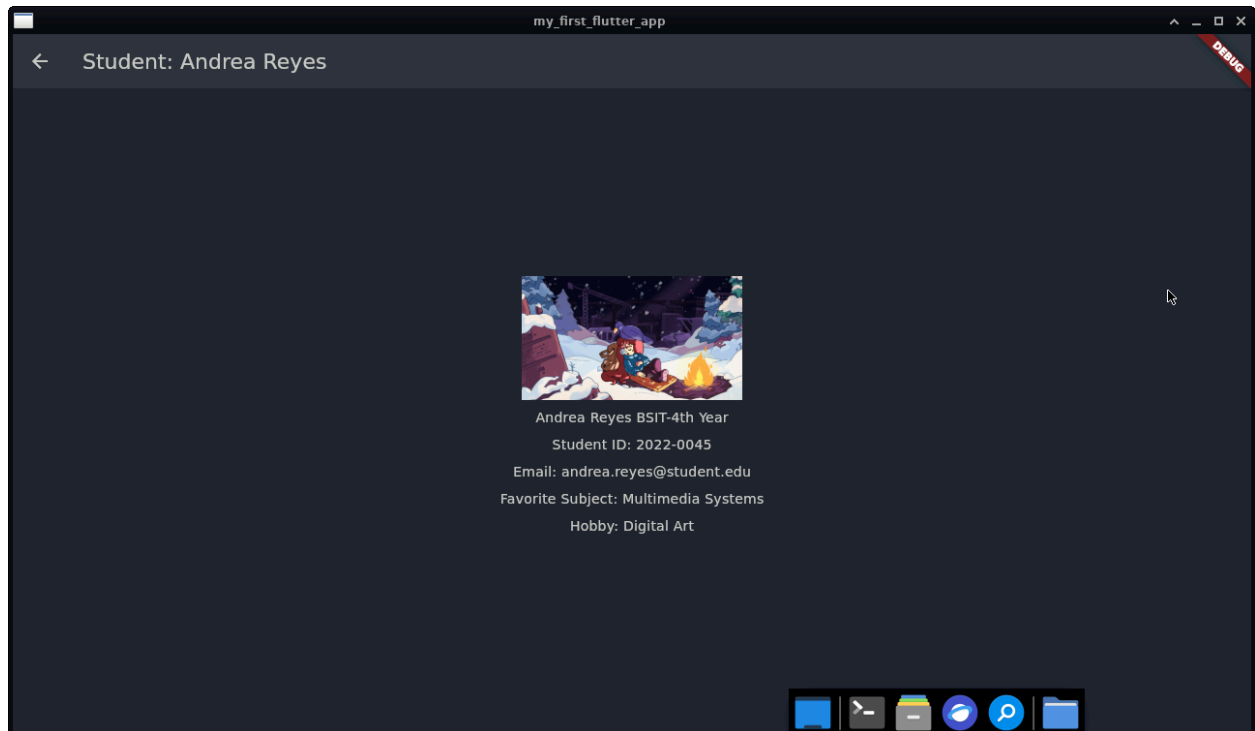
```
+ my_first_flutter_app git:(feat/multi-screen-navigation) ✗ flutter run
Launching lib/main.dart on Linux in debug mode...
Building Linux application...
✓ Built build/linux/x64/debug/bundle/my_first_flutter_app
Syncing files to device Linux... 40ms

Flutter run key commands.
r Hot reload. 📱
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen.
q Quit (terminate the application on the device).

A Dart VM Service on Linux is available at: http://127.0.0.1:37191/_TsuGtSL0A=/
The Flutter DevTools debugger and profiler on Linux is available at: http://127.0.0.1:37191/_TsuGtSL0A=/devtools/?ui=ws://127.0.0.1:37191/_TsuGtSL0A=/ws
Lost connection to device.
+ my_first_flutter_app git:(feat/multi-screen-navigation) ✗
```







🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Multi-Screen Student Directory.

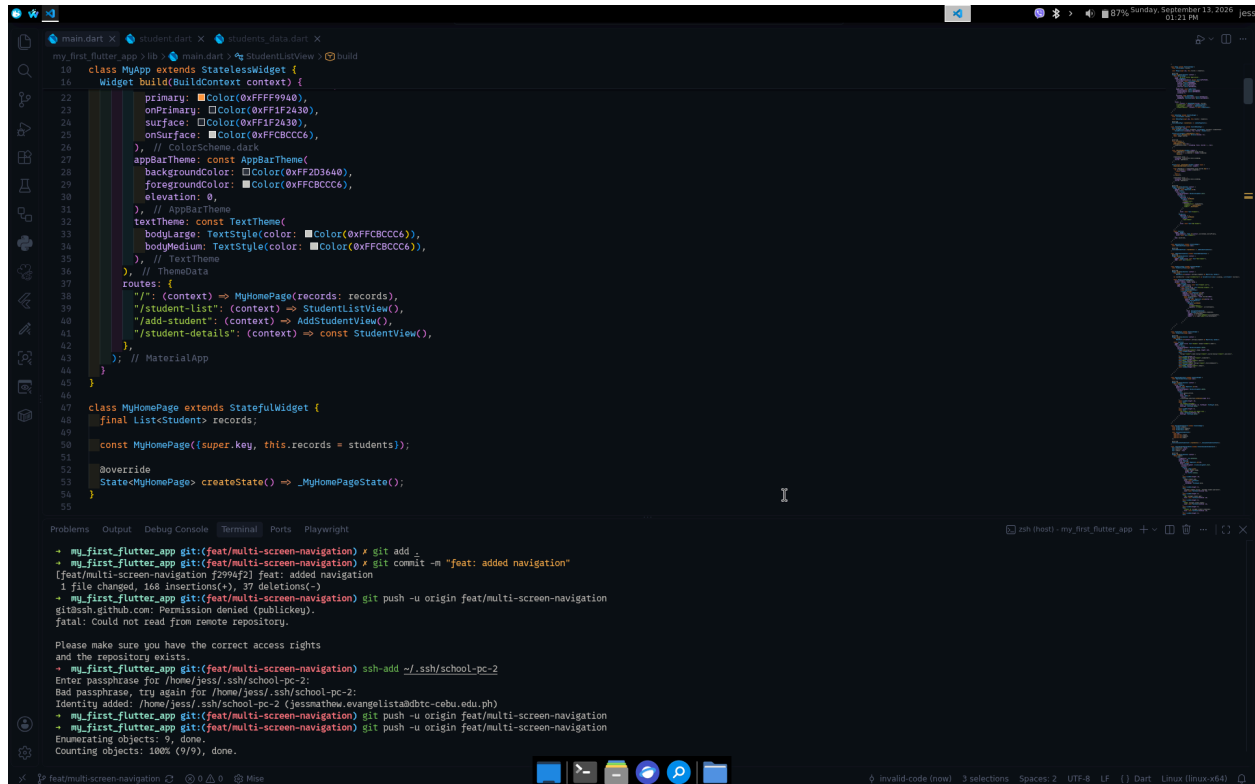
Use a proper Git commit message.

Branch:

feature/multi-screen-navigation

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit



The screenshot shows an IDE with a Dart file named `main.dart` and a terminal window. The Dart code defines a Flutter application with a `MyApp` class extending `StatelessWidget`. It includes a `build` method that returns a `MaterialApp` with a `routes` map. The routes map includes `"/": (context) => MyHomePage(records: records),` `"/student-list": (context) => StudentListView(),` `"/add-student": (context) => AddStudentView(),` and `"/student-details": (context) => const StudentView(),`. The `MyHomePage` class extends `StatefulWidget` and has a `createState` method that returns `_MyHomePageState()`.

The terminal window shows the following commands and output:

```
+ my_first_flutter_app git:(feat/multi-screen-navigation) x git add :
+ my_first_flutter_app git:(feat/multi-screen-navigation) x git commit -m "feat: added navigation"
[feat/multi-screen-navigation f29042] feat: added navigation
1 file changed, 168 insertions(+), 37 deletions(-)
+ my_first_flutter_app git:(feat/multi-screen-navigation) git push -u origin feat/multi-screen-navigation
git push github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
+ my_first_flutter_app git:(feat/multi-screen-navigation) ssh-add ~/.ssh/school-pc-2
Enter passphrase for /home/jess/.ssh/school-pc-2:
Bad passphrase, try again for /home/jess/.ssh/school-pc-2:
Identity added: /home/jess/.ssh/school-pc-2 (jessmathew.evangelista@dtc-ccu.edu.ph)
+ my_first_flutter_app git:(feat/multi-screen-navigation) git push -u origin feat/multi-screen-navigation
+ my_first_flutter_app git:(feat/multi-screen-navigation) git push -u origin feat/multi-screen-navigation
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
```